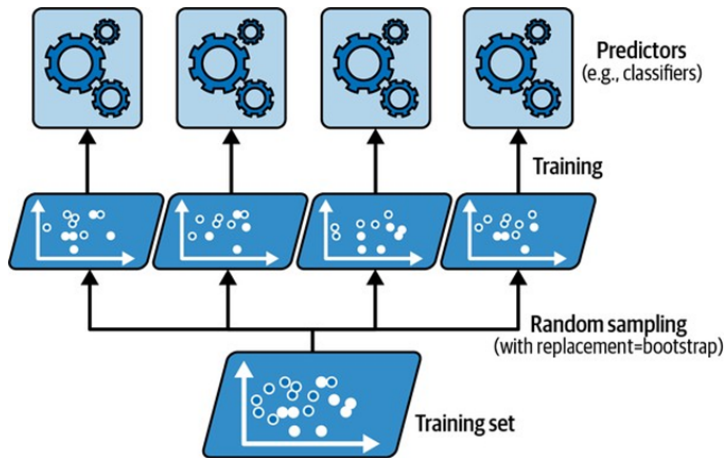


Đặc điểm của Bagging và Pasting

- Hàm tổng hợp cuối cùng thường là **chế độ thống kê** (đối với phân loại) hoặc **giá trị trung bình** (đối với hồi quy).
- Mỗi bộ dự đoán riêng lẻ có **độ lệch (bias) cao hơn** so với khi huấn luyện trên toàn bộ dữ liệu.
- Nhưng kết hợp lại, tập hợp có độ lệch tương tự nhưng **phương sai (variance) thấp hơn** rất nhiều.
- **Mở rộng tốt:** Các bộ dự đoán độc lập nên có thể huấn luyện song song (tận dụng đa lõi CPU hoặc cụm máy chủ).

Minh họa: Quá trình lấy mẫu Bagging/Pasting



Hình 7-4. Túi hóa và Dán nhãn trên các mẫu ngẫu nhiên

Bagging và Pasting trong Scikit-Learn

- Scikit-Learn cung cấp lớp `BaggingClassifier` (hoặc `BaggingRegressor`).
- **Tham số quan trọng:**
 - `n_estimators`: Số lượng bộ dự đoán (ví dụ: 500 cây).
 - `max_samples`: Kích thước tập mẫu ngẫu nhiên cho mỗi bộ.
 - `bootstrap=True`: Sử dụng Bagging (hoặc `False` cho Pasting).
 - `n_jobs=-1`: Sử dụng tất cả các lõi CPU để tăng tốc.

Mã nguồn: BaggingClassifier

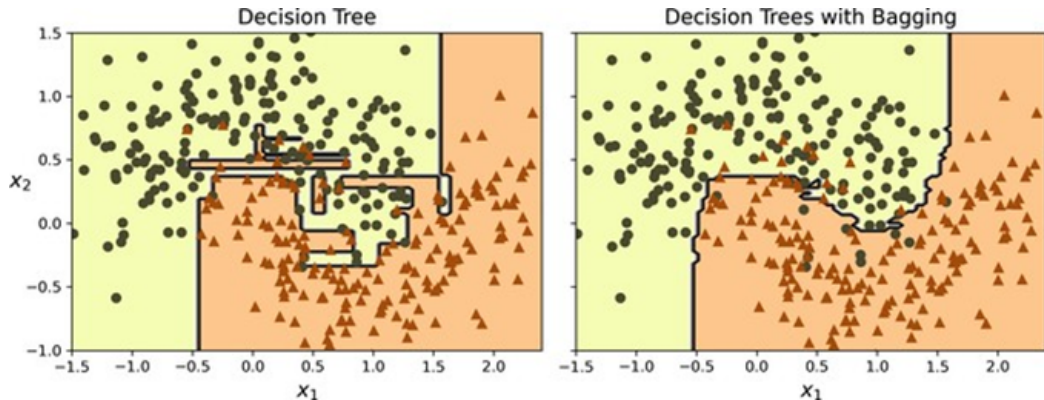
Huấn luyện tập hợp gồm 500 cây quyết định bằng Bagging:

```
1 from sklearn.ensemble import BaggingClassifier
2 from sklearn.tree import DecisionTreeClassifier
3
4 bag_clf = BaggingClassifier(
5     DecisionTreeClassifier(),
6     n_estimators=500,
7     max_samples=100,
8     bootstrap=True, # Doi thanh False de dung Pasting
9     n_jobs=-1,
10    random_state=42
11 )
12 bag_clf.fit(X_train, y_train)
13
```

Phân tích kết quả của Bagging

- Bagging mang lại nhiều đa dạng hơn trong các tập con huấn luyện, do đó Bagging có độ lệch nhỏ hơn một chút so với Pasting.
- Sự đa dạng này giúp các bộ dự đoán ít tương quan hơn → **phương sai của tập hợp được giảm mạnh hơn.**
- Nhìn chung, Bagging thường mang lại mô hình tổng quát hóa tốt hơn và là lựa chọn ưu tiên mặc định.

So sánh Cây đơn lẻ và Tập hợp túi hóa



Hình 7-5. Cây quyết định đơn lẻ (trái) vs. Tập hợp Bagging 500 cây (phải)

Đánh giá ngoài mẫu (Out-of-Bag Evaluation)

- Trong Bagging, việc lấy mẫu có hoàn lại (bootstrap) khiến một số trường hợp được lấy nhiều lần, số khác thì không được lấy.
- Thống kê cho thấy: mỗi bộ dự đoán chỉ lấy trung bình khoảng **63%** số trường hợp.
- Khoảng **37%** trường hợp không bao giờ được bộ dự đoán đó nhìn thấy gọi là các trường hợp **ngoài mẫu (Out-of-Bag - OOB)**.
- Có thể dùng 37% OOB này để đánh giá (evaluate) tập hợp mà **không cần tập xác thực riêng biệt**.

Mã nguồn: Đánh giá OOB

Yêu cầu Scikit-Learn tính toán điểm OOB tự động bằng cách bật tham số
oob_score=True:

```
1 bag_clf = BaggingClassifier(  
2     DecisionTreeClassifier(), n_estimators=500,  
3     oob_score=True, n_jobs=-1, random_state=42  
4 )  
5 bag_clf.fit(X_train, y_train)  
6  
7 # Xem diem OOB  
8 print(bag_clf.oob_score_) # Output: 0.896  
9  
10 # Kiểm tra trên tập Test thực tế  
11 from sklearn.metrics import accuracy_score  
12 y_pred = bag_clf.predict(X_test)  
13 print(accuracy_score(y_test, y_pred)) # Output: 0.92  
14
```

Random Patches và Random Subspaces

- Lớp BaggingClassifier hỗ trợ lấy mẫu theo **đặc trưng (features)** thông qua `max_features` và `bootstrap_features`.
- **Random Patches:** Lấy mẫu cả *trường hợp huấn luyện* và *đặc trưng*. Rất hiệu quả cho dữ liệu nhiều chiều (chẳng hạn như hình ảnh).
- **Random Subspaces:** Giữ toàn bộ trường hợp huấn luyện (`bootstrap=False`, `max_samples=1.0`) nhưng *chỉ lấy mẫu ngẫu nhiên đặc trưng*.
- Đổi một chút độ lệch cao hơn để lấy phương sai thấp hơn.

Rừng ngẫu nhiên (Random Forests)

- Rừng ngẫu nhiên là một tập hợp các cây quyết định, thường được huấn luyện bằng phương pháp **Bagging** với `max_samples` bằng toàn bộ kích thước tập huấn luyện.
- Thuật toán Rừng ngẫu nhiên **đưa thêm sự ngẫu nhiên** vào việc phát triển cây: thay vì tìm kiếm đặc trưng tốt nhất trong toàn bộ đặc trưng, nó tìm kiếm **đặc trưng tốt nhất trong một tập con ngẫu nhiên** của các đặc trưng (mặc định là $\sqrt{n}$ đặc trưng).
- Dẫn đến sự đa dạng cây lớn hơn, đổi độ lệch cao hơn lấy phương sai thấp hơn, mang lại một mô hình tổng quát hóa tốt.

Mã nguồn: RandomForestClassifier

Sử dụng lớp RandomForestClassifier tiện lợi và tối ưu hơn so với dùng BaggingClassifier lồng DecisionTreeClassifier:

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 rnd_clf = RandomForestClassifier(
4     n_estimators=500,
5     max_leaf_nodes=16,
6     n_jobs=-1,
7     random_state=42
8 )
9 rnd_clf.fit(X_train, y_train)
10 y_pred_rf = rnd_clf.predict(X_test)
11
```

So sánh BaggingClassifier và RandomForestClassifier

Mã lệnh BaggingClassifier sau đây gần tương đương với RandomForestClassifier trước đó:

```
1 bag_clf = BaggingClassifier(  
2     DecisionTreeClassifier(max_features="sqrt", max_leaf_nodes=16),  
3     n_estimators=500,  
4     n_jobs=-1,  
5     random_state=42  
6 )  
7
```

- RandomForestClassifier được tối ưu hiệu năng tốt hơn nhiều cho cây quyết định.

Cây ngẫu nhiên cực đại (Extra-Trees)

- Có thể làm cho cây ngẫu nhiên hơn nữa bằng cách sử dụng các **ngưỡng ngẫu nhiên** cho mỗi đặc trưng thay vì tìm ngưỡng chia cắt tốt nhất có thể.
- Tập hợp gồm các cây cực kỳ ngẫu nhiên này được gọi là **Extra-Trees** (Extremely Randomized Trees).
- Lợi ích: Tăng tốc độ huấn luyện đáng kể do không phải tốn kém thời gian tìm ngưỡng tốt nhất, đổi thêm một chút độ lệch lấy phương sai thấp hơn nữa.

Cài đặt Extra-Trees trong Scikit-Learn

Trong `DecisionTreeClassifier`, đặt tham số `splitter="random"` để tạo cây Extra-Tree đơn lẻ. Hoặc sử dụng tập hợp trực tiếp bằng `ExtraTreesClassifier`:

```
1 from sklearn.ensemble import ExtraTreesClassifier
2
3 extra_clf = ExtraTreesClassifier(
4     n_estimators=500,
5     max_leaf_nodes=16,
6     n_jobs=-1,
7     random_state=42
8 )
9 # API giống hệt với RandomForestClassifier
10
```

Tầm quan trọng của đặc trưng

- **Feature Importances:** Rừng ngẫu nhiên rất tuyệt vời để đo lường tầm quan trọng tương đối của từng đặc trưng.
- Scikit-Learn đo lường bằng cách tính toán mức độ mà các nút cây sử dụng một đặc trưng để **làm giảm độ không tinh khiết** (impurity), tính trung bình trên toàn bộ rừng.
- Kết quả được chuẩn hóa để tổng độ quan trọng của tất cả các đặc trưng bằng 1.
- Thuộc tính chứa kết quả: `feature_importances_`.